Chapter #7

Code Shape

Difference between physical register and virtual register?

Physical register:

- A named register in the target ISA.
- In memory-to-memory model, the IR typically uses physical register names.

Virtual register:

- In register-to-register model, the compiler assumes that it has enough registers to express the computation.
- It invents a distinct name, a virtual register, for each value that can legally reside in a register.

Date: Day: M T W T F S

Q. What is purpose of memory model?

- Compilation options affect the placement.
- for example, code compiled to work with a debugger should preserve all values that the debugger can name - typically named variables.
- The compiler must decide, for each value, whether to keep it in a register or to keep it in memory.
- In general, compilers adopt a "memory model"
  ↳ set of rules to guide it in choosing locations for values.
- Two common policies:
  ⇒ memory-to-memory model
  ⇒ Register-to-Register model

⇒ The choice between them has a major impact on the code that the compiler produces.

Date: Day: M T W T F S

Q. What is memory-to-memory and register-to-register model?

In compiler design, these two terms refers to the strategies used for instruction selection and code generation.

Register-to-register model:

- The aim of this model is to minimize the use of memory by keeping most variables in registers.
- it involves using registers for intermediate values and computations, optimizing for speed and efficiency.

Memory-to-Memory model:

- it relies more on accessing variables directly from memory.
- it may generate code that frequently loads and stores values between memory and registers.

Q What is Rvalue and Lvalue?

Rvalue

- An expression evaluated to a value is an rvalue.

Lvalue

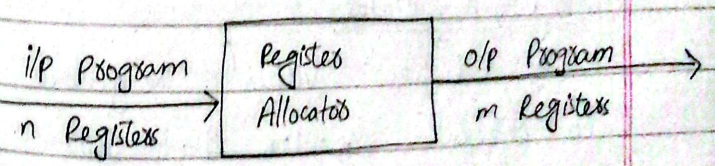
- An expression evaluated to a location is an lvalue.

Q. What is predicated execution?

- An architecture feature in which some operations take a boolean-valued operand that determines whether or not the operation takes effect.
- Architectures that support predicated execution let the compiler avoid some conditional branches.

Registration Allocation in Code Generation :-

- Registers are the fastest locations in the memory hierarchy.
- But this resource is limited. It comes under the most contained resources of the target processor.
- Register allocation is an NP-complete problem.
- However, this problem can be reduced to graph coloring to achieve allocation and assignment.
- Therefore a good register allocator computes an effective approximate solution to a hard problem.



Date: Day: M T W T F S

Register allocator:

- It determines which values will reside in the register and which register will hold each of those values.
- It takes as its input a program with an arbitrary number of registers and produces a program with a finite register set that can fit into the target machine.

Allocation vs Assignment:

Allocation:

- Maps an unlimited namespace onto that register set of the target machine.

Reg-to-Reg model:

- Maps virtual registers to physical register but spills excess amount to memory

Date: Day: M T W T F S

Mem-to-Mem Model:-

Maps some subset of the memory location to a set of names that models the physical register set.

⇒ Allocation ensures that code will fit the target machine's reg set at each instruction.

Assignment:-

- Maps an allocated name set to the physical register set of the target machine.
- Assumes allocation has been done so that code will fit into the set of physical registers.
- No more than 'k' values are designated into the registers. where 'k' is the no of physical registers.

Date: Day: M T W T F S

Local Register Allocation and Assignments

- Allocation just inside a basic block is called Local Reg. Allocation.
- Two approaches for local reg. allocation:
  - ⇒ Top Down approach
  - ⇒ Bottom up approach

Global register Allocation and Assignments

- Global register allocation and assignment aim to optimize the performance of compiled programs by strategically assigning variables to processor registers.

Global register allocation considers the entire program and aims to allocate registers to variables in a way that maximize the utilization of registers.

⇒ To make the decision about allocation and assignments, the global allocator mostly uses graph coloring by building and interfare graph.

Main Issues of register allocator:

⇒ The main issue of a register allocator is minimizing the impact of spill code.

- Execution time for spill code
- Code space for spill operation
- Data space for spilled values